Go2 Continu Rebounce Setup and Experiment Plan

1 Current Task

The current task is continuous rebounce control for Go2. At reset, each environment samples a scalar target apex-height command

$$h^* \sim \mathcal{U}(0.5, 0.8) \text{ m}, \tag{1}$$

and independently samples the initial drop height

$$h_{\text{drop}} \sim \mathcal{U}(0.5, 0.8) \text{ m}. \tag{2}$$

The robot base is placed at h_{drop} with zero root velocity and the default joint state. The policy must keep rebounding for the full episode. The objective is not to jump higher forever, nor merely to recover to the initial release height, but to make each rebound apex return close to the commanded target height while maintaining a flat quadruped posture and staying in place.

Unlike the earlier single-rebounce setup, reaching an apex is no longer a termination condition. The episode continues until either a failure condition is met or the 20 s time limit is reached. A successful rollout should therefore show repeated cycles of

fall, support interaction, rebound, valid apex, re-arm after the feet return near the support.

2 Environment Variants

Two environment IDs are used:

ID	Scene	Notes
`Go2-Rebounce-Flat`	rigid plane	4096 envs, env spacing 2.5 m
`Go2-Rebounce-Trampoline`	deformable trampoline	2048 envs, env spacing 4.0 m

The trampoline variant keeps the same rebounce MDP terms but replaces the support with a deformable trampoline and repins the trampoline rim. Task success does not rely on rigid-deformable contact sensors. Apex and foot-clearance gates use only root state and kinematic foot body positions.

3 Timing

The simulator time step and action decimation are

$$\Delta t_{\text{sim}} = 0.005 \text{ s}, \qquad d = 4, \qquad \Delta t = d\Delta t_{\text{sim}} = 0.02 \text{ s}. \tag{3}$$

The current training episode length is

$$T_{\text{episode}} = 20 \text{ s}. \tag{4}$$

IsaacLab multiplies every weighted reward term by Δt, and logged `Episode_Reward/*` values are additionally divided by T_{episode}.

4 Reset and Command

The rebounce command is

$$\boldsymbol{c} = [h^*]. \tag{5}$$

In the current setup, the target rebound-apex height and the initial drop height are sampled independently from the same range:

$$h^* \sim \mathcal{U}(0.5, 0.8), \qquad h_{\text{drop}} \sim \mathcal{U}(0.5, 0.8). \tag{6}$$

The reset event writes h^* into the command buffer and sets the root state to

$$\boldsymbol{p}_{\text{root}} = \boldsymbol{o}_{\text{env}} + \left[p_x^0,\ p_y^0,\ h_{\text{drop}} + h_{\text{offset}}\right], \qquad \boldsymbol{v}_{\text{root}} = \mathbf{0}, \tag{7}$$

where $h_{\text{offset}} = 0$ in the current config. Joint positions are reset to the default Go2 pose and joint velocities are reset to zero.

During the rollout, the target command is resampled after

$$t_{\text{resample}} \sim \mathcal{U}(10, 20) \text{ s}. \tag{8}$$

This changes only h^*; it does not teleport the robot or change the latched initial drop height h_0. With a 20 s episode, most rollouts see zero or one mid-episode target change, which is enough to test adaptation without turning the task into rapid command following.

The command term stores the following buffers:

Symbol	Buffer	Meaning
h^*	`_target_apex_height`	commanded target apex height
h_0	`drop_height`	root height latched after reset
$\hat{h}_{\text{apex}}$	`last_apex_height`	latched valid-apex root height
$\hat{h}^*_{\text{apex}}$	`last_apex_target_height`	target active at the latched valid apex
b^{apex}	`is_apex`	one-step valid-apex flag
`armed`	`_apex_armed`	next apex can be counted

5 Observations and Actions

Two actor-observation variants are kept for comparison.

Full-state simulation observation. The original full-state actor observation is

$$\boldsymbol{o}_t^{\pi,\text{full}} = \left[h^*,\, \boldsymbol{p}_t^w,\, \boldsymbol{q}_t^w,\, \boldsymbol{v}_t^b,\, \boldsymbol{\omega}_t^b,\, \tilde{\boldsymbol{q}}_t,\, \tilde{\dot{\boldsymbol{q}}}_t,\, \boldsymbol{a}_{t-1}\right]. \tag{9}$$

Here $\boldsymbol{p}_t^w$ is the world root position, $\boldsymbol{q}_t^w$ is the unique world root quaternion, $\boldsymbol{v}_t^b$ is base linear velocity, and $\boldsymbol{\omega}_t^b$ is base angular velocity. This variant is useful for simulation-only baselines and for quantifying how much full-state information helps MLP+DR.

Hardware-realistic deployable observation. The current deployable actor observation removes world root position, world root quaternion, and base linear velocity:

$$\boldsymbol{o}_t^{\pi,\text{deploy}} = \left[h^*,\, \boldsymbol{g}_t^b,\, \boldsymbol{\omega}_t^b,\, \tilde{\boldsymbol{q}}_t,\, \tilde{\dot{\boldsymbol{q}}}_t,\, \boldsymbol{a}_{t-1}\right], \tag{10}$$

where $\boldsymbol{g}_t^b$ is the projected gravity vector in the base frame. This observation is closer to what can be deployed without mocap: joint position, joint velocity, IMU-like projected gravity, IMU angular velocity, previous action, and the commanded target apex height. It intentionally does not expose absolute base height or vertical velocity to the actor.

The deployable actor observation uses additive noise on projected gravity, base angular velocity, joint position, and joint velocity. The critic is asymmetric and keeps privileged full-state terms without corruption:

$$\boldsymbol{p}_t^w, \qquad \boldsymbol{q}_t^w, \qquad \boldsymbol{v}_t^b.$$

Thus the critic can use full simulator state during training, while the actor remains deployable. The policy action is a 12-DoF joint-position target action with the default Go2 joint offset. In the trampoline variant, a separate action term keeps the trampoline rim pinned.

6 Apex Event and Metrics

A valid apex is detected by a root vertical velocity sign change gated by the apex re-arm state and a geometric foot-clearance condition:

$$b_t^{\text{vel}} = (\dot{z}_{t-1} > 0) \wedge (\dot{z}_t \leq 0), \tag{11}$$

$$b_t^{\text{feet+}} = \mathbb{I}\left[\min_{i \in \mathcal{F}} z_{i,t}^{\text{local}} > z_{\text{surface}} + 0.08\right], \tag{12}$$

where $\mathcal{F}$ is the set of four Go2 feet and $z_{\text{surface}} = 0$ in the local support frame. In the implementation this flag is named `feet_above_clearance`.

To avoid counting multiple vertical velocity zero-crossings around the same physical apex, the apex detector is armed only once per bounce. After a valid apex is counted, it is disarmed. It is re-armed only when the feet return below the same clearance threshold:

$$b_t^{\text{feet}-} = \mathbb{I}\left[\max_{i \in \mathcal{F}} z_{i,t}^{\text{local}} \leq z_{\text{surface}} + 0.08\right]. \tag{13}$$

In the implementation this re-arm flag is named `feet_below_clearance`. Thus an apex counted by the metric/reward/timeout logic is the valid apex

$$b_t^{\text{valid}} = \texttt{armed}_t \wedge b_t^{\text{vel}} \wedge b_t^{\text{feet+}}. \tag{14}$$

The command term is the owner of this apex state machine. When b_t^{valid} fires, it latches the apex height and the target active at that instant as `last_apex_height` and `last_apex_target_height`. The target is latched because command resampling can occur after event detection and before the delayed reward reads the event. Reward and termination terms consume this command-owned event one manager step later; this avoids maintaining duplicate apex detectors in reward and termination code.

The logged count metrics are:

Metric	Condition	Interpretation
`apex_count`	b_t^{valid}	valid apex count
`height_matched_apex_count`	$b_t^{\text{valid}} \wedge \lvert h_t - h^* \rvert < 0.25$	apex count close to target height
`error_peak_height`	mean $\lvert h_t - h^* \rvert$ over counted apexes	height tracking error

7 Termination

The current termination terms are:

Term	Condition	Role
`base_orientation`	base tilt angle > 0.6 rad	failure
`non_foot_contact`	non-foot rigid contact > 1 N	failure
`no_valid_apex_timeout`	no valid apex for 2 s	failure
`time_out`	20 s reached	successful long rollout if other failures absent

`no_valid_apex_timeout` reads the command-owned valid-apex pulse and resets its timer whenever that pulse is observed. This prevents a standing policy from simply waiting for the long time limit.

8 Rewards

The weighted reward is

$$\begin{aligned} R_t = \Delta t\Big(& -250\, r_t^{\text{fail}} + 50\, r_t^{\text{apex}} \\ & - 1.5 \times 10^{-2}\, r_t^{\text{energy}} \\ & - 2\, r_t^{\text{orient}} - 0.5\, r_t^{\text{inplace}} - 10^{-2}\, r_t^{\text{action}} - 0.05\, r_t^{\text{joint}} - 10\, r_t^{\text{limit}}\Big). \end{aligned} \tag{15}$$

The failure pulse is

$$r_t^{\text{fail}} = \mathbb{I}[\texttt{base\_orientation}] + \mathbb{I}[\texttt{non\_foot\_contact}] + \mathbb{I}[\texttt{no\_valid\_apex\_timeout}]. \tag{16}$$

There is no separate normal-timeout penalty in the current setup. A rollout that reaches `time_out` without triggering a failure termination is treated as a successful long rollout.

For the delayed apex pulse reward, the height error is computed from the latched command event:

$$e_t^{h,\text{apex}} = |\texttt{last\_apex\_height} - \texttt{last\_apex\_target\_height}|. \tag{17}$$

The flat-body gate is

$$g_t^{\text{flat}} = \exp\left(-\frac{\left\|\boldsymbol{g}_{t,xy}^{b}\right\|^2}{0.35^2}\right), \tag{18}$$

The height reward does not include an additional soft foot-clearance gate; foot clearance is enforced only by the valid-apex event detector. The apex pulse reward is

$$r_t^{\text{apex}} = \mathbb{I}[b_{t-1}^{\text{valid}}] \exp\left(-\left(\frac{e_t^{h,\text{apex}}}{0.10}\right)^2\right) g_t^{\text{flat}}. \tag{19}$$

The remaining penalties are standard orientation, action-rate, joint-deviation, and joint-position-limit regularizers. The in-place term penalizes drift from the reset xy position and reset yaw:

$$r_t^{\text{inplace}} = \left\|\frac{\boldsymbol{p}_{xy,t} - \boldsymbol{p}_{xy,0}}{0.25}\right\|^2 + \left(\frac{\text{wrap}(\psi_t - \psi_0)}{0.5}\right)^2. \tag{20}$$

Reward scale. In the current 20 s setup, a single perfect apex pulse contributes

$$50 \cdot 0.02/20 = 0.05$$

to the logged `Episode_Reward/rebounce_height`. A single failure termination contributes approximately

$$-250 \cdot 0.02/20 = -0.25$$

for the environments where that failure happened, before averaging over all reset environments. With the current absolute-work weight, a typical sparse energy pulse of $W^{\text{abs}}/h^* \approx 250$ contributes

$$-1.5 \times 10^{-2} \cdot 250 \cdot 0.02/20 \approx -0.00375$$

to the logged `Episode_Reward/energy_penalty` for one valid apex.

9 Implementation Files

File	Purpose
`go2_rebounce_env_cfg.py`	scene, reset, reward, termination config
`mdp/commands.py`	rebounce command, apex metrics, re-arm state, energy metrics
`mdp/events.py`	reset from sampled drop height
`mdp/rewards.py`	height reward, energy penalty, and failure pulses
`mdp/terminations.py`	timeout-without-apex termination
`scripts/rsl_rl/play-rebounce.py`	play script with apex height and energy printout

10 Experiment Roadmap

The experiment order should separate task-definition changes from style, efficiency, and final ablations.

10.1 Step 0: Freeze the Current Continuous-Rebounce Baseline

First train and save a clean baseline with the current config. Record the git state, run ID, checkpoint, and videos. The expected healthy indicators are:

- `Episode_Termination/time_out` close to 1,
- `no_valid_apex_timeout` close to 0,
- `apex_count` close to the visually observed bounce count,
- `height_matched_apex_count`/`apex_count` high,
- `error_peak_height` low.

10.2 Step 1: Decouple Initial Drop Height from Target Apex Height

This has been implemented in the current setup. The target height and the initial release height are no longer the same sampled scalar:

$$h^* \sim \mathcal{U}(0.5, 0.8), \qquad h_{\text{drop}} \sim \mathcal{U}(0.5, 0.8).$$

The height rewards and metrics now compare apex height against h^* rather than against h_{drop}. This tests whether the policy controls rebound height, rather than only recovering to the height from which it was dropped.

10.3 Step 2: Add In-Place Hopping

This has also been implemented in the current setup with a small dense penalty on displacement from the reset xy position and reset yaw. Suggested diagnostics for the run are

$$\|\boldsymbol{p}_{xy,t} - \boldsymbol{p}_{xy,0}\|, \qquad \|\boldsymbol{v}_{xy,t}\|, \qquad |\psi_t - \psi_0|, \qquad |\dot{\psi}_t|.$$

The current weight is intentionally modest so vertical rebounce quality is not suppressed. The goal is to keep the robot rebounding in place without sacrificing apex-height tracking.

10.4 Step 3: Minimal Leg-Motion Regulation

Before optimizing energy, first remove obvious high-frequency or visually unacceptable leg motion. This should be a light regulation pass, not a strong posture-imitation objective. The goal is to prevent wasted actuator motion and unsafe leg swings while still allowing the robot to shape its body and legs to use the trampoline.

Test the following regularization variants one at a time:

1. increase `joint_deviation_l1` weight, e.g. $-0.05 \rightarrow -0.10$ first; avoid jumping immediately to a strong posture penalty because it can suppress useful trampoline-exploitation strategies;
2. add a small joint-velocity penalty to reduce fast flight leg motion;
3. test whether the command-side hard foot-clearance apex gate is still needed after removing the reward-side soft foot-clearance gate.

This step should be done before energy optimization so that the energy term is not merely compensating for a leg-motion artifact. After each variant, check `apex_count`, `height_matched_apex_count`, `error_peak_height`, in-place drift, and videos. If height tracking or survival degrades, the regularization is too strong.

10.5 Step 4: Add Energy Optimization on a Fixed Trampoline

Only after the behavior is stable and visually acceptable, add energy terms on the fixed trampoline. Keeping trampoline parameters fixed during this step isolates the question: can the policy maintain commanded apex height with less active work by using the trampoline's passive energy storage?

Two mechanical-work definitions should be tested:

$$P_t^{+} = \sum_j \max(\tau_{j,t}\dot{q}_{j,t}, 0), \qquad P_t^{\text{abs}} = \sum_j |\tau_{j,t}\dot{q}_{j,t}|.$$

The reward is sparse. Between two valid apex events, the energy command accumulates

$$W_k^{+} = \sum_{t \in k} P_t^{+}\Delta t, \qquad W_k^{\text{abs}} = \sum_{t \in k} P_t^{\text{abs}}\Delta t.$$

At the next valid apex, the reward penalty is normalized by the commanded target height h_k^*:

$$c_k^{\text{energy}} = \frac{W_k}{\max(h_k^*, \epsilon)}.$$

Using the target height for the reward prevents the policy from jumping higher than requested only to dilute the work-per-height penalty. The reported metrics below still use the achieved apex height, which keeps them interpretable as the actual work needed per meter of realized rebound height.

The implementation exposes this through the `mode` parameter of `energy_penalty` in `go2_rebounce_env_cfg.py`. Set `mode=positive` for W_k^+/h_k^* or `mode=absolute` for W_k^{abs}/h_k^*. The raw reward follows the same convention as the apex-height pulse: Isaac Lab applies the global Δt scaling, and the event scale is handled in the configured weight. With the current $\Delta t = 0.02$ s, the current absolute-work trial weight is

$$w_{\text{energy}} = -1.5 \times 10^{-2}.$$

The positive-work term penalizes active mechanical energy injected by the motors:

$$\tau_{j,t}\dot{q}_{j,t} > 0.$$

It is the preferred first test because it discourages the robot from actively "kicking" to create apex height while still allowing negative work for landing stabilization. The absolute-work term penalizes both positive and negative mechanical work, so it also discourages active braking:

$$r_t^{\text{abs\_work}} = r_t^{\text{pos\_work}} + \sum_j \max(-\tau_{j,t}\dot{q}_{j,t}, 0).$$

It should be tested after the positive-work variant, or with a much smaller weight, because too much absolute-work penalty can reduce useful damping and postural stabilization during trampoline compression.

A torque-squared term,

$$\sum_j \tau_{j,t}^2,$$

is also useful as a regularizer, but it mainly discourages large torques rather than directly measuring active mechanical work.

The previous positive-work run shows the intended first-order behavior: the logged `Episode_Reward/energy_penalty` is roughly 10% of `Episode_Reward/rebounce_height`, the valid apex count increases, and `positive_work_per_height` decreases. This indicates that the energy term is influencing the policy without dominating the height-tracking task.

Use a small weight or a curriculum that turns on the energy penalty only after the policy already achieves stable rebounce. Keep the height and survival terms dominant. If energy decreases while `apex_count`, `height_matched_apex_count`, `error_peak_height`, or `time_out` degrades, the energy weight is too large or was introduced too early. Compare policies with positive-work-per-height and absolute-work-per-height; use braking ratio as a diagnostic. A low positive-work policy with high braking ratio may simply be absorbing trampoline energy through active braking rather than using the trampoline efficiently.

The energy command term logs:

Metric	Definition
`Metrics/energy/positive_work_per_height`	mean over valid apexes of W_k^+/h_k^{apex}
`Metrics/energy/absolute_work_per_height`	mean over valid apexes of $W_k^{\text{abs}}/h_k^{\text{apex}}$
`Metrics/energy/braking_ratio`	$W_{\text{episode}}^-/\max(W_{\text{episode}}^{\text{abs}}, \epsilon)$

10.6 Step 5: Add Trampoline Parameter Randomization

After the energy-optimized policy works on the fixed trampoline, introduce trampoline randomization. The current Go2 rebounce trampoline setup uses log-uniform Young's modulus randomization over a 0.5× to 2.0× stiffness range while keeping trampoline mass fixed:

$$\log E \sim \mathcal{U}\left(\log(4.0 \times 10^4), \log(1.6 \times 10^5)\right), \qquad m \sim \mathcal{U}(10, 10).$$

This corresponds to $E_0 = 8.0 \times 10^4$ and $E/E_0 \in [0.5, 2.0]$. Thus the current setup is a first-stage stiffness-randomized, fixed-mass experiment.

The randomizable trampoline parameters should be introduced in stages:

Parameter	Meaning	Suggested use
E / `youngs_modulus`	membrane stiffness; controls soft versus stiff rebound timing	first-stage randomization
m / `mass`	trampoline inertia; changes response speed and energy exchange timing	first-stage randomization
`elasticity_damping` or `damping_scale`	material energy dissipation	second-stage, one damping knob at a time
`dynamic_friction`	tangential foot–trampoline friction, affecting slip and in-place stability	robustness sweep
ν / `poissons_ratio`	deformation mode and volume preservation	late-stage small-range sweep

Keep numerical and sleep parameters fixed during the main physics randomization study: `vertex_velocity_damping`, `sleep_damping`, `sleep_threshold`, `settling_threshold`, solver iteration counts, mesh resolution, `contact_offset`, and `rest_offset`. These parameters can affect stability and numerical dissipation, but they are less direct descriptions of a real trampoline than stiffness, mass, damping, friction, and Poisson ratio. For visual parameter sweeps, the rebounce play script exposes fixed overrides for E, m, `dynamic_friction`, `elasticity_damping`, `damping_scale`, and `poissons_ratio`.

Use the current range as the first-stage domain-randomization test:

$$\log E \sim \mathcal{U}(\log(0.5E_0), \log(2.0E_0)), \qquad m = m_0.$$

If this range is too difficult, narrow it temporarily, for example to $E/E_0 \in [0.8, 1.25]$, to separate learning instability from adaptation failure. If the first-stage stiffness randomization is stable, add mass randomization next, e.g. $m \sim \mathcal{U}(0.8m_0, 1.2m_0)$, where the mass term should be used only after a smoke test confirms that runtime mass writes change trampoline dynamics. Randomizing too many parameters too early can make it difficult to tell whether a behavior change came from energy optimization, leg regulation, or trampoline dynamics. It can also encourage a more conservative active-jumping strategy that is robust but less clearly trampoline-efficient.

The randomized parameters should not be provided directly to the deployable actor policy, since the real robot does not know the true trampoline stiffness, mass, or damping. For the paper narrative, the adaptation methods should be tested in increasing order of complexity:

Method	Description	Purpose
MLP + DR	train one memoryless policy across randomized trampoline stiffness; no trampoline parameters and no history in the actor observation	baseline robust policy
MLP + history	stack recent observations/actions and feed the flattened history to an MLP	test whether short-term interaction history is enough
RNN policy	use an LSTM/GRU policy to infer hidden trampoline dynamics from longer temporal context	test recurrent online adaptation
RMA / latent estimator	train with privileged dynamics information, then learn a deployable latent estimator from history	final sim-to-real adaptation story

Privileged trampoline parameters may be used by the critic, teacher, or adaptation module during training, but not as direct actor inputs for a policy intended to transfer to hardware.

The first method is intentionally a partially observable baseline. Since the actor sees neither the trampoline parameter nor interaction history, it cannot condition its timing or leg impedance on the current stiffness. The expected failure mode is therefore an averaged strategy: performance on the soft trampoline, e.g. $E = 4.0 \times 10^4$, may improve compared with a policy trained only at $E = 8.0 \times 10^4$, but nominal-stiffness performance may degrade and the hard trampoline,

e.g. $E = 1.6 \times 10^5$, may require a different timing compromise. This creates the justification for the later methods: history, recurrence, or an RMA-style latent estimator should recover stiffness-dependent behavior while keeping the deployable actor free of privileged trampoline parameters.

Preliminary fixed-condition result. A first evaluation sweep compared two policies: a stiffness-randomized policy trained with

$$E \sim \text{LogUniform}(4.0 \times 10^4, 1.6 \times 10^5)$$

and a fixed-trampoline policy trained only at $E = 8.0 \times 10^4$, $m = 10$ kg. Both policies were evaluated on fixed conditions

$$E \in \{4.0 \times 10^4, 8.0 \times 10^4, 1.6 \times 10^5\}, \qquad h^* \in \{0.5, 0.65, 0.8\}.$$

The stiffness-randomized MLP achieved 100% success on all nine conditions and maintained low height error, with overall

$$\text{MAE} = 0.014 \text{ m}, \qquad |\text{bias}| = 0.011 \text{ m}.$$

The fixed-8.0×10^4 policy performed well at and above its training stiffness, but failed to generalize to the soft trampoline. In particular, at $(E, h^*) = (4.0 \times 10^4, 0.5)$ it had 0% success due to `no_valid_apex_timeout`, and at $(E, h^*) = (4.0 \times 10^4, 0.65)$ it systematically under-jumped with

$$\text{MAE} = 0.103 \text{ m}, \qquad h/h^* = 0.842.$$

This supports the first-stage story that domain randomization improves robustness to trampoline stiffness. However, the randomized policy is not strictly better in all metrics: compared with the fixed policy, it uses more motor work per target height and has higher orientation RMS on the nominal and stiff trampolines. Averaged over all fixed evaluation settings, the randomized policy had

$$W^+/h^* \approx 159, \quad W^{\text{abs}}/h^* \approx 247, \quad \text{orientation\_rms} \approx 0.092,$$

while the fixed policy had

$$W^+/h^* \approx 138, \quad W^{\text{abs}}/h^* \approx 207, \quad \text{orientation\_rms} \approx 0.063.$$

Thus the current MLP+DR policy is a robust average strategy: it solves height tracking across stiffness, but it appears less energy-efficient and slightly less posture-stable than a policy specialized to the nominal trampoline.

Observation-ablation hypothesis for RNN/RMA. The current actor observes root position and base linear velocity. These terms make the instantaneous observation relatively informative: root height and vertical velocity expose bounce phase, impact timing, and part of the trampoline response. This may explain why MLP+DR already achieves strong height tracking despite not observing the trampoline parameter directly. If the goal is to demonstrate the value of online dynamics inference, a useful additional experiment is to reduce instantaneous observability and compare adaptation methods under the same observation constraint.

The proposed controlled variants are:

Variant	Actor observation	Purpose
Full-observation MLP+DR	current observation, including root position and base linear velocity	strong memoryless baseline
Reduced-observation MLP+DR	remove base linear velocity first; optionally also remove root position or at least root height	test whether instantaneous state is sufficient
Reduced-observation history MLP	same reduced observation, with stacked history/actions	test short-horizon implicit velocity/dynamics inference
Reduced-observation RNN/RMA	same reduced deployable observation, with recurrent state or learned latent dynamics	test online trampoline identification

This ablation should be framed carefully. It should not artificially remove sensors only to make the baseline fail. It is justified if real deployment does not provide reliable base position or base linear velocity, or if mocap/state estimator signals are noisy and delayed. In that case, removing or heavily corrupting these observations is a realistic sim-to-real constraint. The fair comparison is then not "full-observation MLP versus reduced-observation RNN", but rather MLP, history-MLP, RNN, and RMA under the same deployable observation set. If reduced-observation MLP+DR loses height tracking while RNN/RMA recovers it, the paper can claim that temporal inference or latent dynamics estimation is needed for robust trampoline adaptation. If height tracking remains good even after the reduction, then RNN/RMA should be judged mainly on energy efficiency, posture stability, and adaptation speed.

To make different trampoline dynamics visible in evaluation, run fixed evaluation sweeps over several discrete trampoline settings instead of relying only on random training averages. For each setting, compare height error, apex count, time-out rate, motor positive work, torque norm, contact/stance duration, and videos. A useful result should show that the policy adapts its timing, leg motion, and motor work across soft and stiff trampolines while maintaining the same commanded apex-height objective.

For paper-level comparison, evaluate each method on fixed conditions rather than only on the training distribution. A condition is a pair such as $(E, h^*) = (4.0\times10^4, 0.5)$, where the trampoline stiffness and target apex height are held fixed for each 20 s rollout. Run N independent rollouts per condition. Compute metrics within each rollout first, then report the mean and standard deviation across rollouts, so that an episode with many apexes does not dominate the statistics.

Metric	Definition	Purpose
`success_rate`	fraction of rollouts that reach the 20 s time-out without `base_orientation`, `non_foot_contact`, or `no_valid_apex_timeout`	stability
`failure_distribution`	counts or fractions of each failed-termination reason	failure diagnosis
`apex_count_per_20s`	number of valid apexes in one rollout	sustained hopping frequency
`height_matched_apex_count`	number of valid apexes satisfying the height tolerance	useful successful hops
`height_mae`	$K^{-1}\sum_{k=1}^{K}\lvert h_k - h_k^*\rvert$ over valid apexes in the rollout	absolute tracking accuracy
`height_rmse`	$\sqrt{K^{-1}\sum_{k=1}^{K}(h_k - h_k^*)^2}$	sensitivity to large errors
`height_bias`	$K^{-1}\sum_{k=1}^{K}(h_k - h_k^*)$	systematic under/over-shoot
`mean_h_over_target`	$K^{-1}\sum_{k=1}^{K} h_k/h_k^*$	normalized achieved height
`height_p90_error`	90th percentile of $\lvert h_k - h_k^*\rvert$	tail robustness
`pos_work_per_target_height`	mean over valid apex intervals of W_k^+/h_k^*	active work cost
`abs_work_per_target_height`	mean over valid apex intervals of W_k^{abs}/h_k^*	total motor work cost
`braking_ratio`	$W^-_{\mathrm{episode}}/\max(W^{\mathrm{abs}}_{\mathrm{episode}}, \epsilon)$	braking versus propulsion balance
`xy_drift`, `yaw_drift`	final or RMS base displacement/yaw error over the rollout	in-place behavior
`orientation_rms`	RMS roll/pitch or projected-gravity error	posture quality

For height metrics, use the valid apexes detected by the command term. If $K = 0$, the rollout should be counted as a failure and the height metrics should be excluded from the successful-rollout mean or reported as undefined. For energy metrics, the target-height denominator is preferred for method comparison because it does not reward jumping higher than commanded; achieved height normalization can still be logged as an efficiency diagnostic.

10.7 TODO: Real-Robot Observation Noise Measurement

Before finalizing sim-to-real observation corruption, measure the observation noise on the physical robot and use the measured ranges to set the policy observation noise. The required measurements are:

- joint position noise from the robot encoder readings;
- joint velocity noise from the robot state estimator or finite differences of encoder readings;
- IMU angular velocity noise for the base angular-velocity observation;
- mocap or state-estimator base position noise;
- mocap or state-estimator base linear-velocity noise.

For each signal, record at least a static standing segment and a representative rebounce or hopping segment. Estimate bias, standard deviation, outliers, latency, and any filtering delay. The resulting noise magnitudes should be used to update the observation corruption in `go2_rebounce_env_cfg.py`, especially for joint position, joint velocity, base angular velocity, base position, and base linear velocity.

10.8 Step 6: Final Ablation Experiments

Run formal ablations after the final task definition is stable. Suggested variants are:

Variant	Change	Main question
Full	final setup	reference
No apex timeout	remove `no_valid_apex_timeout`	is the anti-wait gate needed?
No foot gate	remove the command-side valid-apex foot-clearance gate	does it prevent standing or foot exploits?
No in-place reward	remove xy/yaw terms	does the policy drift?
No regulation	remove flight-leg regulation	does leg motion return?
No energy reward	remove energy term	efficiency versus performance
No trampoline DR	fix trampoline parameters	robustness versus specialization

For every variant, compare success rate, failure distribution, height error, apex count, height-matched apex count, drift metrics, energy, trampoline setting, and videos.

11 Recommended Order

The recommended order is:

1. freeze the current setup with independent drop/target heights and in-place constraints;
2. add only minimal leg-motion regulation needed to remove artifacts;
3. add energy optimization on the fixed trampoline;
4. add trampoline parameter randomization and evaluate discrete trampoline settings;

5. measure real-robot observation noise and update observation corruption;

6. run final ablations.

This order freezes the updated task definition first, removes obvious motion artifacts, optimizes active motor energy on a known trampoline, then tests robustness and adaptation across trampoline dynamics before final explanatory ablations.